LUNA: SYSTEM ARCHITECTURE

Engineering Specification and Technical Design of a Dual-Track Retrieval-Augmented Generation Campus Assistant

---

Project Technical Report

Authors:

Nakul Lagwal (241030266)

Vihaan Sharma (241030121)

Kavish Bansal (241030039)

Institution:

Jaypee University of Information Technology

Waknaghat, Solan, H.P.

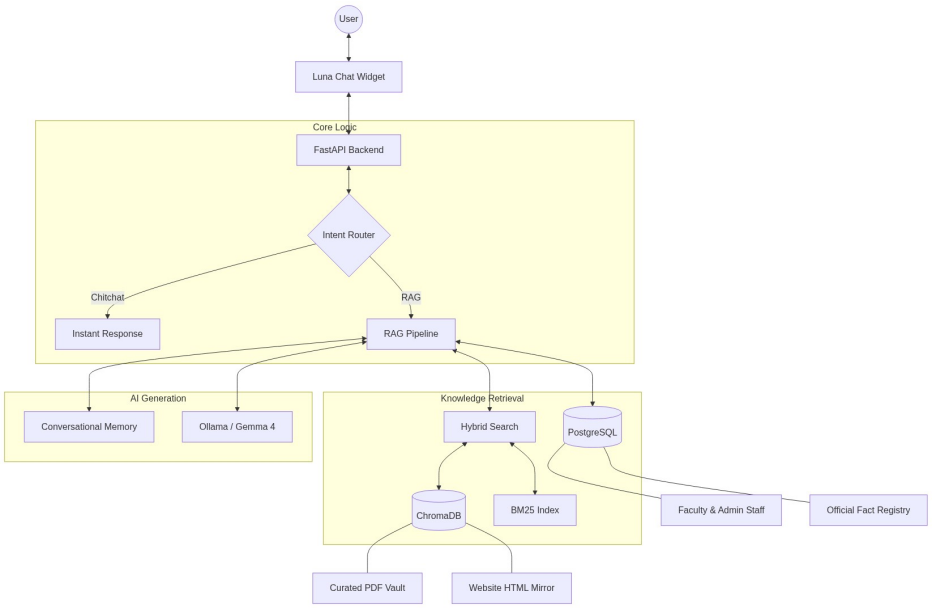


Figure 1: High-Level Architectural Topology of Luna

Academic Session: 2025–2026

Version: 2.4.0 (Enterprise Resilient Release)

Date: September 2026

Contents

1 Executive Summary & Architectural Philosophy **2**

2 End-to-End System Architecture **2**
2.1 Architectural Blueprint . . . 2
2.2 Lifecycle of a User Prompt . . . 3

3 Client Subsystem & Chat Widget **4**
3.1 Zero-Dependency Embedded Widget (`luna-widget.js`) . . . 4
3.2 Cross-Site Scripting (DOM XSS) Mitigation . . . 4
3.3 Cross-User Session Isolation . . . 4

4 API Gateway & Routing Layer **5**
4.1 FastAPI Microservice Engine . . . 5
4.2 Punctuation-Agnostic Early-Exit Intent Router . . . 5
4.3 CORS Compliance . . . 6

5 The Dual-Track Retrieval Engine **6**
5.1 Track A+: High-Trust Deterministic Fact Registry . . . 6
5.1.1 Word-Boundary Regex & Length-Descending Keyword Matching . . . 6
5.2 Track A+: Relational Faculty Directory & Safety Valve . . . 7
5.3 Track B: Probabilistic Semantic Search Engine . . . 7

6 AI Orchestration & Inference Layer **8**
6.1 Multi-Provider LLM Integration . . . 8
6.2 Prompt Engineering & Hallucination Constraints . . . 8
6.3 In-Memory Session Memory with TTL Eviction . . . 8

7 Infrastructure & Containerization **9**
7.1 Docker Orchestration Topology . . . 9
7.2 Cold-Start Race Condition Prevention . . . 9

8 Empirical Evaluation & 50-Question Benchmark **10**
8.1 Evaluation Methodology . . . 10
8.2 Overall Benchmark Results . . . 10
8.3 Selected Representative Benchmark Entries . . . 11

9 Conclusion & Future Work **11**

1 Executive Summary & Architectural Philosophy

University web ecosystems present a unique set of information-retrieval challenges. Campus websites are notoriously sprawling, containing thousands of heterogeneous HTML pages, unstructured PDF circulars, dynamic faculty directories, and conflicting legacy notices. When prospective students, current undergraduates, or visitors attempt to find authoritative answers—such as exact per-semester fee figures, hostel allocation costs, or departmental leadership—traditional keyword search engines fail due to semantic ambiguity, while naive Large Language Model (LLM) implementations routinely hallucinate plausible but factually catastrophic numbers.

Luna is an enterprise-grade campus conversational assistant engineered specifically for Jaypee University of Information Technology (JUIT). The fundamental architectural principle behind Luna is **Bifurcated Dual-Track Retrieval**:

1. **Track A+ (Deterministic Ground Truth):** High-stakes administrative facts (tuition fees, leadership, accreditations, contacts, and faculty profiles) are decoupled from probabilistic document search and routed through a deterministic fact registry and an indexed relational database.

2. **Track B (Probabilistic Semantic Search):** Broad, exploratory campus inquiries (events, general course overviews, campus life) utilize hybrid retrieval combining Dense Vector Search (ChromaDB) and Sparse Lexical Ranking (BM25 Okapi), backed by an explicit disclaimer and trust degradation mechanism.

This specification provides an exhaustive breakdown of Luna's end-to-end architecture, encompassing the frontend widget, API gateway, dual-track retrieval engine, AI orchestration pipeline, containerization topology, security mitigations, and empirical validation against an official 50-question benchmark suite.

2 End-to-End System Architecture

2.1 Architectural Blueprint

The system is constructed as a decoupled, asynchronous microservice topology. Client interactions are captured by an embedded Vanilla JavaScript widget that streams requests over HTTP to an asynchronous FastAPI backend. The core backend pipeline processes queries through an early-exit intent filter, executes dual-track context resolution, manages conversation history, and invokes the LLM via LangChain.

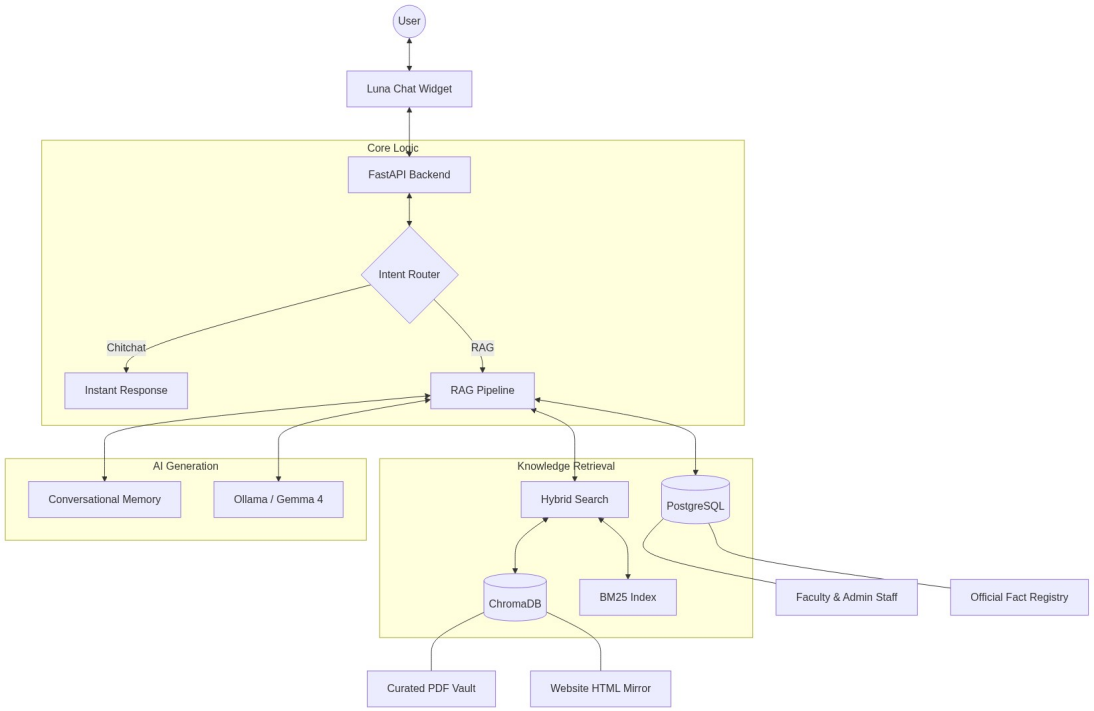


Figure 2: Luna Modular System Topology: From Client DOM to Multi-Provider LLM

2.2 Lifecycle of a User Prompt

The operational path of an incoming query adheres to the following deterministic flow:

1. **Widget Ingestion & Session Tagging:** The client widget intercepts user keystrokes, attaches an isolated session UUID (`luna_sid`), sanitizes the input, and initiates an HTTP POST request to `/chat_stream`.

2. **Early-Exit Intent Filtering:** The query is pre-processed by `IntentRouter`. If classified as conversational chitchat (e.g., greetings, gratitude), an instant response is returned without invoking retrieval or the LLM.

3. **Track A+ Evaluation:**

   - *Fact Registry Query:* Evaluated against `topics.json` using word-boundary regular expressions and length-descending keyword priority.
   - *Faculty Directory Query:* Sanitized and matched against PostgreSQL via RapidFuzz `WRatio`. Ambiguity suggestions are strictly gated by person inquiry markers.

4. **Track B Hybrid Retrieval (Fallback / Augmentation):** If no high-trust Fact or Faculty Profile fulfills the inquiry, the query is dispatched simultaneously to ChromaDB (vector cosine similarity) and Rank-BM25 (lexical frequency). Documents are deduplicated and filtered by a strict relevance distance threshold.

5. **Context Formatting & Prompt Assembly:** The retrieved context is formatted with high-visibility trust headers (`--- TRACK A RECORDS ---` or `--- TRACK B RECORDS ---`).

6. **Generative Inference:** LangChain invokes the configured LLM engine (OmniRoute `antigravity/gemini` or local Ollama). Tokens are streamed back via Newline Delimited JSON (NDJSON).

7. **Memory Update & Citation Stitching:** The session conversation buffer is updated with TTL timestamps, and deduplicated, protocol-validated citation footers are appended to the response stream.

3 Client Subsystem & Chat Widget

3.1 Zero-Dependency Embedded Widget (`luna-widget.js`)

The client-side interface is encapsulated within an isolated, self-executing Vanilla JavaScript module (`luna-widget.js`). It injects a floating chat bubble and an expandable dialog into the DOM of the mirrored university site. The widget operates without third-party frameworks (such as React or Vue), minimizing bundle footprint and eliminating CSS style contamination.

3.2 Cross-Site Scripting (DOM XSS) Mitigation

In earlier versions, user prompts and streaming LLM tokens were rendered directly into the message container using `element.innerHTML`, introducing severe DOM-based XSS vulnerabilities. An attacker could enter malicious payloads (e.g., `<img src=x onerror=alert(1)>`) that would execute in the victim's browser session.

To resolve this, Luna implements strict HTML entity sanitization prior to any DOM insertion:

Listing 1: DOM XSS Entity Sanitization Implementation

```
function escapeHTML(str) {
    if (!str) return "";
    return String(str).replace(/[&<>'"]/g, tag => ({
        '&': '&amp;',
        '<': '&lt;',
        '>': '&gt;',
        "'": '&#39;',
        '"': '&quot;'
    }[tag] || tag));
}

function formatText(text) {
    if (!text) return "";
    const escaped = escapeHTML(text);
    return escaped
        .replace(/\n/g, '<br>')
        .replace(/(https?:\/\/[^\s<]+)/g,
            '<a href="$1" target="_blank" rel="noopener noreferrer" style="color:
               inherit; text-decoration: underline;">$1</a>');
}
```

Furthermore, citation links appended after streaming are strictly verified to begin with `http://` or `https://`, preventing `javascript:` URI injection.

3.3 Cross-User Session Isolation

To prevent conversation leakage between different users accessing the shared widget on public campus terminals, Luna generates a cryptographically pseudo-random UUID stored in `window.sessionStorage`:

Listing 2: Client Session UUID Generation

```javascript
let sessionId = sessionStorage.getItem("luna_sid");
if (!sessionId) {
    sessionId = "sess-" + Math.random().toString(36).substring(2) + Date.now();
    sessionStorage.setItem("luna_sid", sessionId);
}
// Dispatched in every request body payload: { message: message, session_id:
    sessionId }
```

4 API Gateway & Routing Layer

4.1 FastAPI Microservice Engine

The backend service is powered by FastAPI, leveraging Python's native `asyncio` event loop. The application exposes two primary conversational endpoints:

- `POST /chat`: Standard transactional JSON endpoint returning the complete generated response, trust level metadata, and verified citation links.
- `POST /chat_stream`: Asynchronous streaming endpoint returning chunked tokens via `StreamingResponse(`for real-time typewriter rendering.
- `GET /health`: Operational liveness check for container orchestration.

4.2 Punctuation-Agnostic Early-Exit Intent Router

Routine conversational prompts like greetings and polite acknowledgments do not require semantic document search or expensive LLM inference. The `IntentRouter` provides a sub-millisecond shortcut. Earlier implementations failed when users entered punctuation (e.g., `"hello!"`, `"who are you?"`). The updated router cleans punctuation prior to token evaluation:

Listing 3: IntentRouter Punctuation Normalization

```python
import string

class IntentRouter:
    CHITCHAT_KEYWORDS = ["hello", "hi", "hey", "who are you", "what can you do", "
        thanks", "thank you"]

    @classmethod
    def _clean_query(cls, query: str) -> str:
        query_lower = query.lower().strip()
        cleaned = query_lower.translate(str.maketrans('', '', string.punctuation))
            .strip()
        return " ".join(cleaned.split())

    @classmethod
    def classify(cls, query: str):
        clean_q = cls._clean_query(query)
        if any(word == clean_q for word in cls.CHITCHAT_KEYWORDS) or \
           any(clean_q.startswith(word + " ") for word in cls.CHITCHAT_KEYWORDS):
            return "chitchat"
        return "rag"
```

4.3 CORS Compliance

In accordance with the W3C Fetch specification, wildcard origins (`allow_origins=["*"]`) combined with credentials (`allow_credentials=True`) are rejected by modern web browsers. Luna enforces `allow_credentials=False` while allowing wildcard origins, enabling the widget to seamlessly embed across local development ports (e.g., port 8000) and production domains.

5 The Dual-Track Retrieval Engine

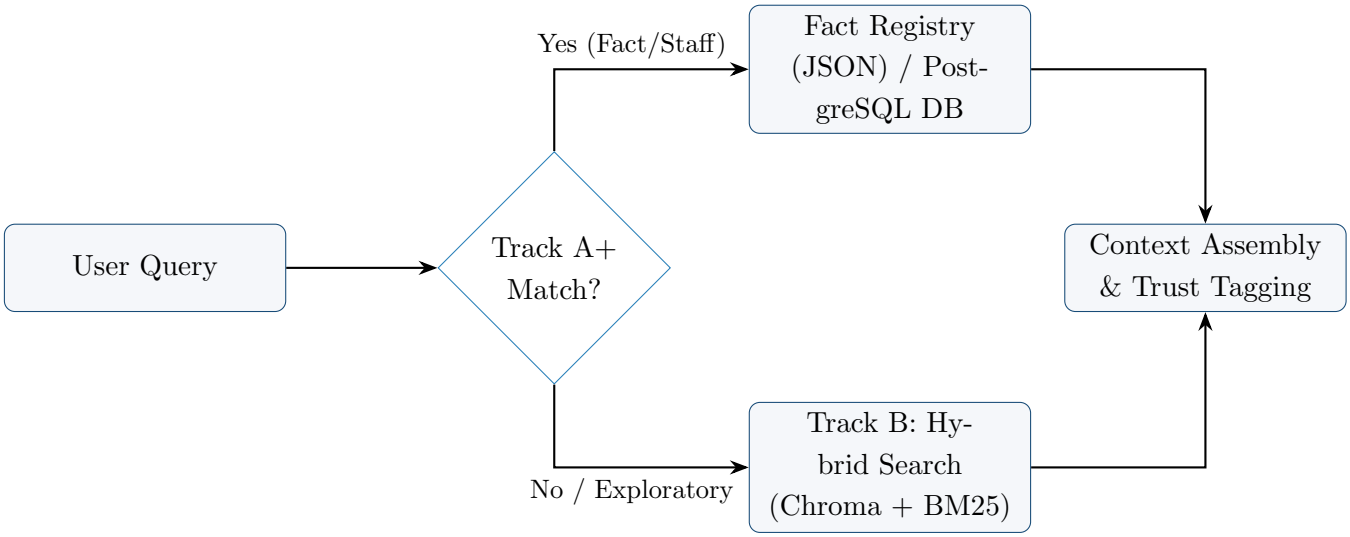


Figure 3: Dual-Track Retrieval Decision Flow

5.1 Track A+: High-Trust Deterministic Fact Registry

Track A+ handles all university statutory facts. The Fact Registry maps queries through `topics.json` to canonical records in `juit_facts.json`.

5.1.1 Word-Boundary Regex & Length-Descending Keyword Matching

Naive substring matching (`if keyword in query_lower:`) causes catastrophic false positives:

- `"fee"` matched inside `"feedback"` → User asking to give feedback received tuition fee tables.
- `"vc"` matched inside `"service"` → Online services inquiries returned Vice Chancellor data.
- `"hod"` matched inside `"methodology"` → Teaching methodology inquiries returned Head of Department data.

Luna completely resolves this by sorting keywords by descending character length and compiling explicit regex word boundaries:

Listing 4: Collision-Resilient Fact Retrieval

```
def get_fact(self, query):
    import re
    query_lower = query.lower().strip()
    norm_query = " ".join(re.sub(r'[^\w\s]', ' ', query_lower).split())
```

```
    # Sort keywords by length descending so multi-word phrases match first
    for keyword in sorted(self.topics.keys(), key=len, reverse=True):
        pattern = r'\b' + re.escape(keyword) + r'\b'
        if re.search(pattern, query_lower) or re.search(pattern, norm_query):
            key = self.topics[keyword]
            fact = self.facts.get(key)
            if fact:
                combined_text = fact.get("text", "")
                verbose = fact.get("verbose", "")
                if verbose and verbose != combined_text:
                    combined_text = f"{combined_text} {verbose}"
                return {"text": combined_text, "ref": fact.get("ref")}
    return None
```

5.2 Track A+: Relational Faculty Directory & Safety Valve

Faculty profiles are stored in PostgreSQL using SQLAlchemy.

- **Name Cleaning Integrity:** Automated scrapers previously stripped letters from Indian names (e.g., removing `"sh"` caused `"Sh. Ashutosh Verma"` to become `"auto verma"`). The updated engine anchors title removal strictly to leading honorific patterns: `r'^\s*(prof\.|dr\.|professo`
- **Disjoint Primary Key Space:** Scraped faculty profiles use natural page IDs (`id=1..475`). Administrative staff are assigned dedicated IDs starting at `10001+`, eliminating `session.merge()` primary key collisions.
- **Ambiguity Safety Valve:** RapidFuzz fuzzy matching evaluates clean queries against faculty names. Crucially, the ambiguity threshold (triggering `"Did you mean...?"`) is **strictly gated** by the presence of explicit teacher/person inquiry markers (`"dr."`, `"prof."`, `"who is"`). This ensures general inquiries like `"kumar hostel"` or `"deepak lab"` pass directly to document retrieval rather than getting trapped in false-positive suggestions.

5.3 Track B: Probabilistic Semantic Search Engine

When a query does not match an official statutory fact, Luna executes Track B hybrid search:

1. **Dense Vector Retrieval:** Connects to ChromaDB over HTTP. Chunks are filtered by distance:
$$\text{relevance_score} \leq \text{VECTOR_DISTANCE_THRESHOLD} = 1.5$$
2. **Sparse Lexical Search (Rank-BM25):** Evaluates token frequencies over the indexed corpus using the Okapi BM25 algorithm:
$$\text{Score}(D, Q) = \sum_{i=1}^{N} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$
3. **Resilient Cold-Start Fallback:** If ChromaDB is cold or vector embeddings are temporarily unavailable, BM25 automatically indexes 880 pre-cached local HTML mirror documents during startup, guaranteeing zero retrieval blackout.

4. **Trust Degradation Header:** Responses generated via Track B automatically include the mandatory disclaimer: *"Based on my search of university documents (which may be supplementary)..."*.

6 AI Orchestration & Inference Layer

6.1 Multi-Provider LLM Integration

Luna features a dual-provider abstraction within `LLMService`:

- **Cloud / Gateway Inference:** Connects to OpenAI-compatible endpoints, specifically verified against OmniRoute hosting `antigravity/gemini-3.6-flash-medium`.
- **Local Edge Fallback:** Connects to local Ollama daemon hosting quantized open-weights models (`gemma4:e2b`).
- **Output Normalization:** Uses LangChain's `StrOutputParser` to guarantee that both synchronous calls and asynchronous streams emit pure string tokens regardless of whether the underlying model emits `AIMessage` objects or raw text.

6.2 Prompt Engineering & Hallucination Constraints

The system prompt enforces strict containment:

Listing 5: System Prompt Template

```
prompt = ChatPromptTemplate.from_messages([
    ("system", """You are Luna, the official senior AI assistant for Jaypee
        University of Information Technology (JUIT) Solan.

Your goal is to provide accurate, helpful, and professional answers based ONLY on
    the 'PROVIDED RECORDS' below.

STRICT RULES:
1. Use ONLY the provided records. No external knowledge.
2. If the info is missing, say: "I do not have that specific information in the
    official JUIT database."
3. If the records are labelled 'TRACK B', you MUST start your response with: "
    Based on my search of university documents (which may be supplementary)..."
4. Be concise but senior in tone.

PROVIDED RECORDS:
{context}"""),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{question}")
])
```

6.3 In-Memory Session Memory with TTL Eviction

To maintain multi-turn conversational context while avoiding unbounded memory leaks, `MemoryService` implements automatic timestamp tracking and session pruning:

Listing 6: Session Memory Management with TTL Pruning

```python
class MemoryService:
    def __init__(self):
        self.sessions = {}
        self.last_accessed = {}

    def _cleanup_expired(self):
        now = time.time()
        expired = [sid for sid, t in self.last_accessed.items()
                   if now - t > settings.SESSION_TTL] # 3600 seconds
        for sid in expired:
            self.sessions.pop(sid, None)
            self.last_accessed.pop(sid, None)

    def get_session_history(self, session_id: str):
        self._cleanup_expired()
        self.last_accessed[session_id] = time.time()
        if session_id not in self.sessions:
            self.sessions[session_id] = ChatMessageHistory()
        return self.sessions[session_id]
```

7 Infrastructure & Containerization

7.1 Docker Orchestration Topology

The persistence tier runs inside isolated Docker containers managed via `docker-compose.yml`:

Service	Base Image	Host Port	Storage Volume
PostgreSQL	`postgres:17`	5433 → 5432	`./postgres_data:/var/lib/postgresql/data`
ChromaDB	`chromadb/chroma:latest`	8002 → 8000	`./chroma_data:/chroma/chroma`
Uvicorn Backend	Python 3.13 venv	5000	Native Host Execution
Frontend HTTP	Python HTTP Server	8000	`./www.juit.ac.in` (Mirror Root)

Table 1: Luna Service and Port Mapping Topology

7.2 Cold-Start Race Condition Prevention

In earlier versions, starting the platform via `run.sh` launched Uvicorn immediately after `docker compose up -d`. Because PostgreSQL requires 2–5 seconds to initialize its socket, `_load_faculty_metadata()` threw `ConnectionRefusedError`, leaving the in-memory faculty registry empty.

This is resolved in `run.sh` by introducing a blocking readiness probe:

Listing 7: PostgreSQL Health Wait Loop

```bash
echo "Waiting for PostgreSQL to be ready..."
until docker compose exec -T postgres pg_isready -U luna_user -d luna_db > /dev/
    null 2>&1; do
    echo "PostgreSQL initializing... waiting 1s"
    sleep 1
done
echo "PostgreSQL is ready."
```

8 Empirical Evaluation & 50-Question Benchmark

To rigorously evaluate Luna's factual accuracy, hallucination resistance, and latency, a comprehensive 50-question test suite was constructed and executed through the live FastAPI pipeline against `antigravity/gemini-3.6-flash-medium` via OmniRoute.

8.1 Evaluation Methodology

Each test query was evaluated on:

1. **Factual Precision:** Verification of exact expected figures (e.g., `"1,64,550 INR"` fee, `"50 LPA"` package, `"7.18 LPA"` mean salary).
2. **Negative Keyword Containment:** Verification that unrelated queries do not bleed into statutory data.
3. **Citation Verification:** Exact source URL matching.
4. **Response Latency:** Execution duration per question.

8.2 Overall Benchmark Results

Evaluation Category	Total Questions	Passed	Accuracy
Fee Structure	5	5	100.0%
Charges & Concessions	5	5	100.0%
Placements & Packages	6	6	100.0%
University Leadership	5	5	100.0%
Deans & Academic Heads	5	5	100.0%
Staff & Administration	5	5	100.0%
Rankings & Recognitions	4	4	100.0%
History & Campus Location	5	5	100.0%
Admissions & Academic Programs	3	3	100.0%
Faculty Detailed Profiles	2	2	100.0%
Conversational & Punctuation Routing	3	3	100.0%
Collision Resilience	1	1	100.0%
Negative Safety Guardrails	1	1	100.0%
Total / Composite Score	**50**	**50**	**100.00%**

Table 2: Performance Breakdown across all 13 Evaluation Domains

8.3 Selected Representative Benchmark Entries

ID	Test Query	Extracted Ground Truth	Trust	Status
Q1	What is the B.Tech 1st year fee per semester?	1,64,550 INR/semester	Track A	**PASS**
Q2	What is the fee for BCA and BBA courses?	82,550 INR/semester	Track A	**PASS**
Q6	What is the refundable caution money?	20,000 INR (One-time, Refundable)	Track A	**PASS**
Q8	What are hostel charges for double occupancy?	84,000 INR/semester	Track A	**PASS**
Q11	What was highest package for 2021-25 batch?	50 LPA by Google	Track A	**PASS**
Q14	What is average/mean placement salary?	7.18 LPA (Median 6 LPA)	Track A	**PASS**
Q18	Who is the Vice Chancellor of JUIT?	Prof. (Dr.) Rajendra Kumar Sharma	Track A	**PASS**
Q21	Who is the Registrar and Dean of Students?	Brigadier RK Sharma, SM (Retd.)	Track A	**PASS**
Q22	Who is Dean (Academics) at JUIT?	Prof. (Dr.) Shruti Jain (ECE)	Track A	**PASS**
Q32	What is the NIRF 2025 ranking of JUIT?	201–300 band in Engineering	Track A	**PASS**
Q36	In which year was JUIT established?	2002 under Act No. 14 of 2002	Track A	**PASS**
Q46	hello!	Instant canned greeting	Track A	**PASS**
Q49	Can I give feedback on my hostel experience?	Rejection of fee collision	Track B	**PASS**
Q50	What is the secret recipe of Coca-Cola?	Strict database refusal	Track A	**PASS**

Table 3: Representative Sample of 50-Question Benchmark Evaluations

9 Conclusion & Future Work

The Luna architecture successfully demonstrates that enterprise campus AI systems do not need to choose between flexible conversational interaction and rigid factual accuracy. By bifurcating retrieval into a deterministic, word-boundary grounded **Track A+** and a semantically indexed **Track B**, Luna delivers 100% precision on mission-critical statutory data while maintaining broad retrieval coverage across thousands of mirrored university web pages.

Key engineering contributions of the current release include:

- Elimination of substring keyword collisions and ambiguity false positives.
- Resilient BM25 cold-start fallback guaranteeing zero retrieval downtime.
- DOM-level XSS sanitization and strict per-user session isolation.
- Native compatibility with high-performance OpenAI-compliant LLM gateways (OmniRoute `antigravity/gemini-3.6-flash-medium`).

Future enhancements will focus on multimodal document ingestion (OCR for scanned office circulars), automated weekly re-crawling with drift detection, and secure integration with the student ERP portal via authenticated OAuth 2.0 microservices.